

## 1. this Keyword

The this keyword in C# is used to refer to the current instance of a class. It resolves ambiguity between instance variables and method parameters or is used to invoke other constructors.

### Common Uses of this:

**Referencing Current Object:** Access instance variables when they are shadowed by method parameters.

Java

```
class Employee {
    string name;

    public Employee(string name) {
        this.name = name; // Refers to the instance variable
    }
}
```

### Invoke Current Class Methods:

Java

```
class Example {
    void MethodOne() {
        Console.WriteLine("Method One");
        this.MethodTwo(); // Calls another method in the same class
    }

    void MethodTwo() {
        Console.WriteLine("Method Two");
    }
}
```

### Invoke Current Class Constructor (Constructor Chaining):

Java

```
class Student {
    string name;
```

```
int age;

public Student(string name) : this(name, 18) { }

public Student(string name, int age) {
    this.name = name;
    this.age = age;
}
}
```

## 2. static Keyword

The static keyword in C# is used to indicate that a member (variable, method, block, or class) belongs to the class rather than any specific object. Static members are shared across all instances of the class.

### Key Features of static:

- **Class-Level Scope:** Shared among all objects.
- **Memory Management:** Stored in a separate memory section to reduce redundancy.
- **Access Without Object:** Can be accessed using the class name directly.
- **Shared State:** Any changes affect all instances.

### Usage of static:

#### 1. Static Variables:

```
Java
class Counter {
    static int count = 0; // Static variable

    public Counter() {
        count++;
    }

    public void DisplayCount() {
        Console.WriteLine("Count: " + count);
    }
}
```

## 2. Static Methods:

```
Java
class MathUtility {
    public static int Square(int x) {
        return x * x;
    }
}

class Program {
    static void Main() {
        Console.WriteLine(MathUtility.Square(5)); // Output: 25
    }
}
```

## 3. Static Constructor:

```
Java
class Example {
    static int value;
    static Example() {
        value = 10;
        Console.WriteLine("Static constructor executed!");
    }
    public static void ShowValue() {
        Console.WriteLine("Value: " + value);
    }
}
```

## 4. Static Nested Class:

```
Java
class Outer {
    public static class Inner {
        public static void Display() {
            Console.WriteLine("Static nested class");
        }
    }
}
```

```
}  
  
class Program {  
    static void Main() {  
        Outer.Inner.Display();  
    }  
}
```

### Advantages of static:

- Memory efficiency by sharing resources.
- Global access to members.
- Ideal for utility functions like Math.Sqrt().

### Limitations of static:

- Cannot override static methods.
- Cannot access instance members without an object.
- May cause thread safety issues if shared across threads.

---

## 3. final (sealed) Keyword

In C#, sealed is used instead of final to prevent a class from being inherited and to restrict method overriding.

### Usage of sealed:

#### 1. Sealed Classes:

```
sealed class Constants {  
    public const double PI = 3.14159;  
}
```

#### 2. Sealed Methods:

```
class Parent {  
    public sealed void Display() {  
        Console.WriteLine("Sealed Method");  
    }  
}
```

```
}  
}
```

```
class Child : Parent {  
    // Cannot override Display()  
}
```

## 4. is Operator (Equivalent to instanceof in Java)

The is operator in C# is used to check if an object is of a specific type.

### Syntax:

object is ClassName

### Example:

```
Java  
class Parent { }  
class Child : Parent { }  
  
class Program {  
    static void Main() {  
        Parent obj = new Child();  
  
        if (obj is Child) {  
            Console.WriteLine("obj is an instance of Child");  
        }  
        if (obj is Parent) {  
            Console.WriteLine("obj is also an instance of Parent");  
        }  
    }  
}
```

### Use Cases:

- **Type Checking in Polymorphism:** Ensures valid type before casting.
- **Avoids Invalid Type Casting:** Prevents InvalidCastException.

## Comparison Table

| Feature             | this                                            | sealed                                | is                        | static                                        |
|---------------------|-------------------------------------------------|---------------------------------------|---------------------------|-----------------------------------------------|
| <b>Purpose</b>      | Refers to the current object.                   | Prevents modification or inheritance. | Checks object type.       | Creates shared class-level members.           |
| <b>Scope</b>        | Current instance of the class.                  | Methods or entire class.              | Objects and types.        | Variables, methods, constructors, or classes. |
| <b>Common Usage</b> | Resolving name conflicts, constructor chaining. | Constants, prevent overriding.        | Polymorphism type checks. | Utility methods, shared properties.           |